

Fachkonzept

Backend-Architektur Reflecta

AI-driven Journal & Reflection Assistant

Technische Spezifikation

Version 1.0

Stand: Mai 2026

Verfasser: Bilal El Hasnaoui (9333793), Simon Lademann (4898928)

Dokumentkontext

Dieses Fachkonzept beschreibt die Backend-Architektur der Anwendung Reflecta. Es richtet sich an Entwickler, Software-Architekten, Product Owner sowie technische Stakeholder und dient als verbindliche Referenz für Implementierung, Wartung und Erweiterung des Systems.

Grundlage des Dokuments sind das bereitgestellte Quellrepository (FastAPI-Backend in Python 3.9) sowie das ergänzend gelieferte Datenbankmodell (ER-Diagramm und One-Table-Design für DynamoDB). Bei Abweichungen zwischen Modell und Implementierung wird beides dokumentiert und der Soll/Ist-Stand explizit ausgewiesen.

Dokumentstruktur

- Kapitel 1 - 2: Fachliche Zielsetzung und Systemüberblick
- Kapitel 3 - 4: Datenmodell und API-Spezifikation
- Kapitel 5 - 6: Business-Logik, Zustandsmodelle und Datenflüsse
- Kapitel 7: Sicherheits- und Architekturkonzept
- Kapitel 8 - 9: Technische Bewertung, offene Fragen und Annahmen

Verwendete Konventionen

- Fett markierte Begriffe kennzeichnen zentrale Fachtermini.
- Tabellen mit der Überschrift 'Soll/Ist' dokumentieren Abweichungen zwischen Spezifikation und Implementierung.
- Geschätzte oder abgeleitete Inhalte sind explizit als 'Annahme' oder 'Inferiert' gekennzeichnet.

1. Zielsetzung der Anwendung

1.1 Fachlicher Zweck

Reflecta ist eine KI-gestützte Journaling- und Reflexionsanwendung. Sie ermöglicht Benutzern das strukturierte Erfassen täglicher Gedanken, Stimmungen und Ereignisse in Form von Journal-Einträgen, kombiniert mit einer Zielverwaltung (Goal Tracking) und einer KI-gestützten Auswertung emotionaler Muster über die Zeit.

Im Unterschied zu klassischen Tagebuch-Apps verbindet Reflecta drei Säulen: strukturierte Selbstreflexion, datengetriebene Mustererkennung (Sleep, Stress, Sentiment, Social Engagement) und KI-gestützte personalisierte Rückkopplung. Die Anwendung soll Anwendern helfen, Zusammenhänge zwischen Wohlbefindensfaktoren zu erkennen und persönliche Ziele aktiv zu verfolgen.

1.2 Hauptfunktionen

- **Journal-Management:** Erstellen, Lesen, Aktualisieren und Löschen tagesbezogener Einträge mit quantitativen Befindlichkeits-Skalen (Sentiment, Sleep, Stress, Social Engagement; jeweils 1-5).
- **Zielverwaltung:** Pflege persönlicher Ziele mit Kategorie, Priorität, Zieldatum und Fortschritts-Tracking.
- **KI-Anreicherung:** Automatische Analyse von Journal-Einträgen durch ein LLM (Aktivitäten-Extraktion, Sentiment-Tagging, Zielzuordnung, Formatierung).
- **Analytics:** Berechnung von Zeitreihen-Trends, Durchschnittswerten, Korrelationen und KI-generierten Zusammenfassungen.
- **Konversationelle Assistenz:** Kontextbezogener Chatbot mit Zugriff auf Ziele und letzte Einträge sowie geführte Reflexionsfragen.
- **Goal Recommendations:** KI-basierte Empfehlung neuer Ziele auf Basis aktueller Journal-Inhalte.
- **Administrationsbereich:** System-Statistiken zu Nutzerzahl, AI-Aufrufen und Erfolgsraten.

1.3 Zielgruppe

Stakeholder	Beschreibung
Endbenutzer	Privatpersonen mit Interesse an strukturierter Selbstreflexion, mentaler Gesundheit und persönlicher Zielerarbeit.
Administratoren	Operativ verantwortliches Personal mit Zugriff auf Nutzungs- und Performance-Metriken.
Entwickler / Architekten	Technische Stakeholder, die Erweiterungen, Wartung und Integration vornehmen.
Product Owner	Verantwortliche für die fachliche Weiterentwicklung der Plattform.

1.4 Kernprozesse

1.4.1 Journal-Eintrag erfassen und anreichern

Der Benutzer erfasst einen Eintrag mit Titel, Datum, Freitext und vier optionalen Skalen-Werten. Nach dem synchronen Persistieren startet ein Background-Task, der das LLM mit dem Inhalt und den

Zielen des Nutzers aufruft. Das LLM liefert formatierten Inhalt, eine Aktivitätsliste, eine Sentiment-Liste sowie eine Liste verknüpfter Ziel-IDs zurück. Diese werden in den Eintrag zurückgeschrieben.

1.4.2 Ziele führen

Der Benutzer legt Ziele mit Pflichtfeldern Titel, Typ, Kategorie und Priorität sowie optionalen Feldern Beschreibung und Zieldatum an. Ziele können explizit aktualisiert werden, ihr Fortschrittswert (progress) wird im aktuellen Stand jedoch nicht automatisch fortgeschrieben (siehe Kapitel 9, offene Punkte).

1.4.3 Analytics und Insights

Der Benutzer ruft Trends, Durchschnitte, Korrelationen oder eine generative Zusammenfassung für einen wählbaren Zeitraum ab. Korrelationen und Summaries werden mit einer TTL von 24 Stunden im AnalyticsCache zwischengespeichert und im Hintergrund neu berechnet, wenn der Cache nicht aktuell ist.

2. Systemübersicht

2.1 Architekturüberblick

Das Reflecta-Backend folgt einer klassischen, geschichteten Backend-Architektur (Layered Architecture) auf Basis von FastAPI. Eingehende HTTP-Anfragen werden durch Router-Komponenten entgegengenommen, an Service-Schichten delegiert und über CRUD-Funktionen mit der Datenhaltungsschicht verbunden. Querschnittsthemen wie Authentifizierung, AI-Aufrufe und Analytics-Berechnungen sind als eigenständige Service-Module gekapselt.

2.1.1 Schichtenmodell

Schicht	Verantwortung	Artefakte im Code
Presentation / API	HTTP-Routing, Request-/Response-Modelle, Auth-Dependencies, Background-Task-Anstoß	app/routes/*.py
Application / Service	Fachliche Orchestrierung, AI-Integration, Analytics-Berechnung, Caching	app/services/*.py
Domain Model	Datenstrukturen, Validierung, Enums (Pydantic)	app/models/*.py
Persistence / CRUD	Datenbankzugriffe, ORM-Operationen	app/db/crud/*.py, app/db/database.py
Cross-Cutting	Auth (Cognito + JWT), AI-Client, Logging, Utils	app/auth/*, app/services/ai_client.py, app/utils/*

2.2 Technologie-Stack

Bereich	Technologie	Funktion
Web-Framework	FastAPI	ASGI-basiertes REST-Framework mit OpenAPI-Auto-Doc
ASGI-Server	Uvicorn	Produktions-/Entwicklungs-Server
Sprache / Runtime	Python 3.9	Basisstack (siehe Dockerfile)
ORM	SQLAlchemy	Relationales Mapping mit declarative_base
Datenbank (Ist)	SQLite	Dateibasierte Datenbank journal.db
Datenbank (Soll laut ERD)	Amazon DynamoDB	One-Table-Design mit GSI/LSI (siehe Mismatch in Kapitel 3)
Validierung	Pydantic v2	Datenmodelle, Enums, ConfigDict
Authentifizierung	AWS Cognito + JWT (RS256)	OAuth2PasswordBearer mit JWKS-Caching
AI-Integration	OpenAI-kompatibler Client	Pluggable für Gemini / z.ai (env-gesteuert)
Statistik / Numerik	NumPy	Pearson-Korrelation, Moving Averages

Bereich	Technologie	Funktion
Background Processing	FastAPI BackgroundTasks	Asynchrone AI-Anreicherung und Cache-Refresh
Caching (App-Layer)	AnalyticsCache (DB-gestützt) + cachetools TTLCache (JWKS)	24h TTL für Analytics, 1h TTL für JWKS
Container	Docker (python:3.9-slim)	Produktions-Deployment via Dockerfile
API-Test-Tooling	Bruno	.bruno-Sammlung im Repo

2.3 Komponentenübersicht

2.3.1 Backend-Komponenten

- `main.py`: Anwendungs-Bootstrap, CORS-Middleware, Router-Registrierung, Lifespan-Hook für Schema-Erstellung.
- `auth/cognito.py`: Cognito-JWT-Verifikation mit JWKS-Cache (TTL 1h).
- `auth/dependencies.py`: `get_current_user` / `get_current_admin` als FastAPI-Dependencies. Just-in-Time User-Provisioning bei erstem Login.
- `routes/`: Endpoint-Definitionen für `auth`, `journal`, `goal`, `chatbot`, `analytics`, `admin`.
- `db/database.py`: Engine, SessionLocal, SQLAlchemy-Modelle (`User`, `JournalEntry`, `Goal`, `AnalyticsCache`, `AIUsageLog`) und Association-Tabelle `journal_goal_association`.
- `db/crud/`: Separierte CRUD-Operationen für `Journal`, `Goal`, Hilfsfunktionen.
- `services/ai_client.py`: Zentralisierter AI-Adapter mit Provider-Auswahl, Retry-Logik (Exponential Backoff), Usage-Logging.
- `services/gemini_agent.py`: Fachliche AI-Funktionen (`analyze_entry`, `recommend_goals`, `generate_journal_question`, `enhance_goal_description`, `summarize_journal_entries`, `generate_correlation_insights`).
- `services/gemini_chatbot.py`: Kontextbezogener Chatbot mit System-Prompt und Nutzerkontext.
- `services/analytics.py`: `AnalyticsService` mit Methoden für Trends, Averages, Korrelationen, Streak-Berechnung.
- `models/`: Pydantic-Schemas für Eingaben, Ausgaben und Enums.

2.4 Kommunikationsflüsse

2.4.1 Externe Kommunikation

- Client (Web-Frontend localhost:3000 laut CORS-Whitelist) -> Backend via REST/HTTPS.
- Backend -> AWS Cognito IdP via HTTPS (JWKS-Abruf).
- Backend -> AI-Provider (Gemini- oder z.ai-API, OpenAI-kompatibel) via HTTPS.

2.4.2 Interne Kommunikation

- Synchron: HTTP-Request -> Router -> Service/CRUD -> Datenbank -> Response.
- Asynchron via BackgroundTasks: Journal-Anreicherung nach POST/PUT, Cache-Refresh nach Analytics-Abfragen.
- Cross-Cutting: Jeder AI-Aufruf erzeugt einen Eintrag in der Tabelle `ai_usage_logs`.

2.5 Architekturprinzipien

- Separation of Concerns: klare Trennung Routes / Services / CRUD / Models.
- Dependency Injection via FastAPI Depends() für DB-Session und Auth-Kontext.
- Stateless API: keine Server-Sessions, Authentifizierung pro Request über Bearer-Token.
- Domain-orientierte Modulgliederung statt funktional flacher Struktur.
- Provider-Abstraktion: AI-Client erlaubt Wechsel zwischen Gemini und z.ai per Environment-Variable.
- Fail-Fast bei Auth-Fehlern, graceful Degradation bei AI-Fehlern (Logging, keine Request-Blockade).

3. Datenmodell

Hinweis: Architektur-Mismatch zwischen Spezifikation und Implementierung

Das mitgelieferte ER-Diagramm und das One-Table-Design beschreiben einen NoSQL-Ansatz für Amazon DynamoDB (PK/SK-Pattern, GSI/LSI-Indizes, Access Patterns).

Der aktuelle Code (app/db/database.py) implementiert dagegen ein relationales Modell mit SQLAlchemy ORM auf SQLite-Basis (Foreign Keys, Association-Tabelle, declarative_base).

Beide Modelle werden im Folgenden dokumentiert: Abschnitt 3.1 beschreibt den Ist-Stand (SQLAlchemy/SQLite), Abschnitt 3.2 das Soll-Modell laut ERD (DynamoDB). Abschnitt 3.3 vergleicht beide explizit.

3.1 Ist-Datenmodell (SQLAlchemy auf SQLite)

3.1.1 Entitäten im Überblick

Entität	Fachliche Bedeutung
users	Plattform-Benutzer mit Anbindung an AWS Cognito (cognito_sub als externer Identifier).
journal_entries	Tagesbezogene Reflexionseinträge inklusive quantitativer Wohlbefindens-Skalen und KI-generierter Anreicherungen.
goals	Persönliche Ziele eines Nutzers mit Kategorie, Priorität, optionalem Zieldatum und Fortschrittswert.
journal_goal_association	Many-to-many-Verbindungstabelle zwischen Einträgen und Zielen, befüllt durch die KI-Analyse.
analytics_cache	Cache für rechenintensive Auswertungen (Summary, Correlation Insights) je Nutzer und Periode.
ai_usage_logs	Audit-Log aller AI-Aufrufe (Feature, Modell, Token-Nutzung, Erfolgsstatus).

3.1.2 Tabellendefinitionen

Tabelle: users

Attribut	Typ	Constraints	Beschreibung
id	INTEGER	PK, AUTOINC	Interner Primärschlüssel
cognito_sub	STRING	UNIQUE, NOT NULL, INDEX	Externer Identifier aus AWS Cognito (sub-Claim)
email	STRING	UNIQUE, NOT NULL, INDEX	E-Mail-Adresse des Nutzers
is_admin	BOOLEAN	NOT NULL, DEFAULT false	Berechtigung für Admin-Endpoints
created_at	DATETIME	DEFAULT now(UTC)	Erstellungszeitpunkt des User-Records

Tabelle: journal_entries

Attribut	Typ	Constraints	Beschreibung
id	INTEGER	PK, AUTOINC	Interner Primärschlüssel
user_id	INTEGER	FK -> users.id, NOT NULL	Zugehöriger Nutzer
title	STRING	NOT NULL	Titel des Eintrags
date	DATE	NOT NULL	Fachliches Datum (kann vom created_at abweichen)
content	TEXT	NOT NULL	Roh-Text des Eintrags
sentiment_level	INTEGER	NULLABLE, 1-5	Stimmungs-Skala (1 sehr negativ ... 5 sehr positiv)
sleep_quality	INTEGER	NULLABLE, 1-5	Schlafqualität-Skala
stress_level	INTEGER	NULLABLE, 1-5	Stress-Skala
social_engagement	INTEGER	NULLABLE, 1-5	Sozial-Skala (1 allein ... 5 sehr sozial)
formatted_content	TEXT	NULLABLE	KI-formatierte Version des Inhalts
activities	TEXT	NULLABLE	Komma-separierte Aktivitäten (KI-extrahiert)
sentiments	STRING	NULLABLE	Komma-separierte Sentiments (KI-extrahiert)
created_at	DATETIME	DEFAULT now(UTC)	Anlagedatum
updated_at	DATETIME	NULLABLE	Letzte Änderung (gesetzt bei Update + Enrichment)

Tabelle: goals

Attribut	Typ	Constraints	Beschreibung
id	INTEGER	PK, AUTOINC	Interner Primärschlüssel
user_id	INTEGER	FK -> users.id, NOT NULL	Zugehöriger Nutzer
title	STRING	NOT NULL	Titel des Ziels
type	STRING	NOT NULL	Goal-Typ (z. B. 'One-time', 'Recurring')
target_date	DATE	NULLABLE	Optional, vorgesehen für 'One-time'-Ziele
category	STRING	NOT NULL	Fachliche Kategorie (Health, Career, etc.)
priority	STRING	NOT NULL, DEFAULT 'Low'	Werte: 'High' 'Medium' 'Low'
description	TEXT	NULLABLE	Optionale Beschreibung
progress	INTEGER	NOT NULL, DEFAULT 0	Fortschritt 0-100 (aktuell nicht serverseitig automatisch berechnet)

Attribut	Typ	Constraints	Beschreibung
created_at	DATETIME	DEFAULT now(UTC)	Anlagedatum
updated_at	DATETIME	NULLABLE	Letzte Änderung

Tabelle: journal_goal_association (M:N)

Attribut	Typ	Constraints	Beschreibung
journal_id	INTEGER	FK -> journal_entries.id	Referenz auf den Eintrag
goal_id	INTEGER	FK -> goals.id	Referenz auf das Ziel

Hinweis (Inferiert): Die Tabelle besitzt im aktuellen Code keinen expliziten zusammengesetzten Primärschlüssel und keine UNIQUE-Constraint auf (journal_id, goal_id). Mehrfach-Zuordnungen sind technisch möglich. Empfehlung: zusammengesetzten PK ergänzen.

Tabelle: analytics_cache

Attribut	Typ	Constraints	Beschreibung
id	INTEGER	PK, AUTOINC	Interner Primärschlüssel
user_id	INTEGER	FK -> users.id, NOT NULL	Zugehöriger Nutzer
period	STRING	NOT NULL	Periode-Bezeichner (z. B. '30days', '6months')
cache_type	STRING	NOT NULL	'summary' 'correlation_insights'
content	TEXT	NOT NULL	Serialisierter Cache-Inhalt (JSON oder Plaintext)
generated_at	DATETIME	DEFAULT now(UTC)	Erzeugungszeitpunkt (für TTL-Berechnung)

Hinweis (Inferiert): Code enthält keine UNIQUE-Constraint auf (user_id, period, cache_type). Bei wiederholtem Schreiben wird zwar im Code geprüft (Lookup vor Upsert), eine Datenbank-Garantie fehlt jedoch.

Tabelle: ai_usage_logs

Attribut	Typ	Constraints	Beschreibung
id	INTEGER	PK, AUTOINC	Interner Primärschlüssel
user_id	INTEGER	FK -> users.id, NULLABLE	Optional, falls Aufruf nicht user-gebunden
feature	STRING	NOT NULL	Logischer Aufrufkontext (z. B. 'analyze_entry')
model	STRING	NOT NULL	Verwendetes AI-Modell
input_tokens	INTEGER	NULLABLE	Prompt-Tokens
output_tokens	INTEGER	NULLABLE	Completion-Tokens
success	BOOLEAN	NOT NULL, DEFAULT true	Aufrufstatus

Attribut	Typ	Constraints	Beschreibung
error_message	TEXT	NULLABLE	Fehlerbeschreibung im Fehlerfall
created_at	DATETIME	DEFAULT now(UTC)	Zeitstempel des Aufrufs

3.1.3 Relationen und Kardinalitäten

Beziehung	Kardinalität	Beschreibung
users -> journal_entries	1 : N	Ein Nutzer kann beliebig viele Einträge besitzen.
users -> goals	1 : N	Ein Nutzer kann beliebig viele Ziele besitzen.
journal_entries <-> goals	M : N	Über journal_goal_association; KI-getrieben befüllt.
users -> analytics_cache	1 : N	Pro Nutzer mehrere Cache-Einträge (Periode x Typ).
users -> ai_usage_logs	1 : N (optional)	AI-Logs können auch ohne User-Bezug existieren.

3.1.4 Fachliche Bedeutung der zentralen Entitäten

User

Der User ist der zentrale Identitätsträger. Die externe Identifikation erfolgt über cognito_sub aus AWS Cognito. Beim ersten authentifizierten Request wird der User automatisch im lokalen Schema angelegt (Just-in-Time Provisioning in app/auth/dependencies.py).

JournalEntry

Ein JournalEntry ist die primäre Datenquelle für Analytics und KI-Auswertungen. Er besitzt zwei Datenflächen: roh erfasste Inhalte (title, content, Skalen) und KI-angereicherte Inhalte (formatted_content, activities, sentiments, verknüpfte Goals). Die Anreicherung geschieht asynchron, sodass die initiale Response des POST-Endpoints nicht durch LLM-Latenz blockiert wird.

Goal

Ein Goal repräsentiert ein persönliches Ziel mit Lebenszyklus von der Anlage bis zur Zielerreichung. Der Fortschritt wird im Feld progress gehalten, im aktuellen Code aber ausschließlich über explizite Updates verändert. Eine automatische Fortschrittsberechnung anhand verknüpfter Einträge ist nicht implementiert.

AnalyticsCache

Der Cache reduziert AI-Kosten und Antwortzeiten bei wiederholten Analytics-Abfragen. Bei abgelaufener TTL liefert der Endpoint den (alten oder leeren) Cache-Stand sofort zurück und startet die Neuberechnung als Background-Task.

AIUsageLog

Der AIUsageLog ist die Grundlage für Cost-Tracking, Erfolgsraten-Monitoring und Feature-Nutzungsanalysen im Admin-Bereich. Er erfasst sowohl erfolgreiche als auch fehlgeschlagene Aufrufe inklusive Token-Nutzung.

3.2 Soll-Datenmodell laut ERD (DynamoDB One-Table-Design)

Das mitgelieferte ER-Diagramm und das One-Table-Design beschreiben ein NoSQL-Modell, das sämtliche Entitäten in einer DynamoDB-Tabelle ablegt. Die Differenzierung erfolgt über das Schlüsselpaar Partition Key (PK) und Sort Key (SK) sowie über Global und Local Secondary Indices.

3.2.1 Key-Schema

Entität	PK	SK	Wesentliche Attribute
User	USER	<sub>	email, is_admin, created_at
JournalEntry	USER#<sub>	ENTRY#<entryId>	title, date, content, sentiment_level, sleep_quality, stress_level, social_engagement, formatted_content, activities, sentiments, created_at, updated_at
Goal	USER#<sub>	GOAL#<goalId>	title, type, target_date, category, priority, description, progress, created_at, updated_at
Entry->Goal Link	USER#<sub>	ENTRY#<entryId>#GOAL#<goalId>	created_at
Goal->Entry Link	USER#<sub>	GOAL#<goalId>#ENTRY#<entryId>	created_at
AnalyticsCache	USER#<sub>	CACHE#<cache_type>#<period>	content, created_at
AIUsageLog	USER#<sub>	AILOG#<created_at>#<logId>	feature, model, input_tokens, output_tokens, success, error_message, created_at

3.2.2 Sekundärindizes (Soll-Modell)

Local Secondary Indices (LSI)

- LSI1 - Entries by date per user: sk = ENTRY#DATE#<date>
- LSI2 - Goals by category per user: sk = GOAL#CAT#<category>
- LSI3 - Goals by priority per user: sk = GOAL#PRIO#<priority>

Global Secondary Indices (GSI)

- GSI1 - User by email (Login-Pfad): pk = EMAIL#<email>
- GSI2 - Admin / cross-user lookups: pk = ENTITY#<type>, sk = <created_at>#<id>
- GSI3 - AI usage analytics cross-user: pk = AILOG#<feature>

3.2.3 Wesentliche Access Patterns laut Spezifikation

Access Pattern	Query
Nutzer per sub abrufen	pk = USER, sk = <sub>
Nutzer per E-Mail (Login)	GSI1_pk = EMAIL#<email>

Access Pattern	Query
Alle Einträge eines Nutzers	pk = USER#<sub>, begins_with(sk, 'ENTRY#')
Einträge nach Datum (LSI1)	pk = USER#<sub>, LSI1_sk BETWEEN ENTRY#DATE#<d1> AND ENTRY#DATE#<d2>
Ziele nach Kategorie (LSI2)	pk = USER#<sub>, LSI2_sk = GOAL#CAT#<category>
Ziele nach Priorität (LSI3)	pk = USER#<sub>, LSI3_sk = GOAL#PRIO#<priority>
Cache je Nutzer + Typ + Periode	pk = USER#<sub>, sk = CACHE#<cache_type>#<period>
AI-Logs eines Features cross-user	GSI3_pk = AILog#<feature>

3.3 Soll/Ist-Vergleich

Aspekt	Soll (laut ERD)	Ist (laut Code)
Datenbanktyp	Amazon DynamoDB (NoSQL, Single-Table)	SQLite (relational, Multi-Table)
Zugriffsmuster	Predefined Access Patterns mit PK/SK	Generische SQL-Queries via SQLAlchemy
M:N Eintrag-Ziel	Zwei Link-Items (Entry->Goal, Goal->Entry)	Eine Association-Tabelle journal_goal_association
Login per E-Mail	GSI1 mit pk = EMAIL#<email>	User-Lookup via cognito_sub, E-Mail nur zur Anzeige
Admin Cross-Lookups	GSI2 (ENTITY#<type>)	SQLAlchemy Aggregat-Queries (func.count, func.sum)
AI-Logs cross-user	GSI3 (AILog#<feature>)	WHERE-Filter auf ai_usage_logs.feature
Skalierbarkeit	Horizontal, AWS-managed	Begrenzt durch Single-File-DB (SQLite)

Konsequenz: Eine produktive Inbetriebnahme erfordert entweder eine Migration auf DynamoDB samt Umbau der Persistenzschicht (Repository-Pattern, kein ORM) oder eine Anpassung des ERD an die relationale Realität. Beide Varianten sind in Kapitel 8 bewertet.

4. API-Endpoint-Konzept

4.1 Globale API-Eigenschaften

- Base URL (lokal): `http://127.0.0.1:8000`
- OpenAPI / Swagger UI: `http://127.0.0.1:8000/docs`
- Content-Type: `application/json`
- Authentifizierung: OAuth2 Bearer Token (JWT aus AWS Cognito), Verifikation gegen JWKS.
- DEV_MODE: Bei Umgebungsvariable `DEV_MODE=true` wird die Authentifizierung umgangen und ein synthetischer 'dev-user' verwendet (siehe Kapitel 7 / 9, Sicherheitshinweis).
- CORS: Whitelisted Origin `http://localhost:3000`, alle Methoden und Header erlaubt, Credentials aktiviert.
- Fehlerantworten: FastAPI-Standard mit JSON-Body `{ "detail": "..."}.`

4.2 Endpoint-Übersicht

Methode	Pfad	Zweck
GET	/	Healthcheck / Welcome-Message
GET	/auth/me	Aktuelles User-Profil lesen
GET	/journal/entries/	Journal-Einträge listen
GET	/journal/entries/{id}	Einzelnen Journal-Eintrag lesen
POST	/journal/entries/	Journal-Eintrag erstellen (mit AI-Anreicherung)
PUT	/journal/entries/{id}	Journal-Eintrag aktualisieren
DELETE	/journal/entries/{id}	Journal-Eintrag löschen
GET	/goals/	Ziele listen (priorisiert sortiert)
GET	/goals/{id}	Einzelnes Ziel lesen
POST	/goals/	Ziel erstellen
PUT	/goals/{id}	Ziel aktualisieren
DELETE	/goals/{id}	Ziel löschen
POST	/goals/recommend	KI-basierte Zielempfehlungen abrufen
POST	/goals/enhance-description	KI-gestützte Beschreibungs-Verbesserung
POST	/ai/chat/	Kontextueller Chatbot-Dialog
POST	/ai/journal-question/	Reflexionsfrage generieren
GET	/analytics/trends/	Zeitreihen-Trends (Moving Averages)
GET	/analytics/stats/	Durchschnittswerte und Streak
GET	/analytics/correlations/	Korrelationen + KI-Insights
GET	/analytics/summary/	KI-generierte Zusammenfassung der Periode

Methode	Pfad	Zweck
GET	/admin/stats	System-Statistiken (admin-only)

Anmerkung (Inferiert): Die Bruno-Sammlung im Repository (.bruno/) verweist auf einige Pfade, die im Code nicht (mehr) existieren, z. B. /analytics/averages oder /analytics/weekly-patterns. Im Folgenden wird der Code als verbindliche Wahrheit dokumentiert.

4.3 Auth-Endpoints

4.3.1 GET /auth/me - Aktuelles Profil lesen

Feld	Wert
Beschreibung	Liefert das User-Profil des authentifizierten Aufrufers. Erzeugt den User just-in-time, falls er noch nicht lokal existiert.
HTTP-Methode	GET
Pfad	/auth/me
Authentifizierung	JWT (Bearer)

Response (200)

```
{
  "id": 42,
  "email": "user@example.com",
  "is_admin": false,
  "created_at": "2026-05-22T10:14:33Z"
}
```

HTTP-Statuscodes

Code	Bedeutung
200	Erfolgreiche Verarbeitung
401	Fehlende oder ungültige Authentifizierung
422	Validierungsfehler (Pydantic / FastAPI)
500	Unerwarteter Serverfehler

Fachliche Business-Logik

- Verifikation des Bearer-Tokens gegen die Cognito-JWKS.
- Lookup des Nutzers über cognito_sub.
- Falls nicht vorhanden: INSERT in users (Just-in-Time Provisioning).
- Falls E-Mail im Token aktueller als im DB-Record: Update auf users.email.

4.4 Journal-Endpoints

4.4.1 GET /journal/entries/ - Einträge listen

Feld	Wert
Beschreibung	Listet alle Journal-Einträge des aktuellen Nutzers, sortiert nach date DESC, dann created_at DESC.
HTTP-Methode	GET
Pfad	/journal/entries/
Authentifizierung	JWT

Query-Parameter

Parameter	Typ	Pflicht	Beschreibung
skip	int (>=0)	nein	Offset für Pagination (Default 0)
limit	int (1-100)	nein	Max. Anzahl Treffer (Default 100)

Response (200, Auszug)

```
[
  {
    "id": 7,
    "title": "Reflections on My Day",
    "date": "2026-05-16",
    "content": "Today was productive...",
    "sentiment_level": 4,
    "sleep_quality": 3,
    "stress_level": 2,
    "social_engagement": 3,
    "formatted_content": "**Morning** ...",
    "activities": "Morning run, Reading",
    "sentiments": "Content, Grateful",
    "created_at": "2026-05-16T19:42:01Z",
    "updated_at": "2026-05-16T19:42:18Z",
    "goals": [ { "id": 2, "title": "Take long walks", "..." } ]
  }
]
```

4.4.2 GET /journal/entries/{id} - Einzelnen Eintrag lesen

Feld	Wert
Beschreibung	Lädt einen Eintrag inklusive zugeordneter Ziele. 404, wenn der Eintrag nicht zum Nutzer gehört.
HTTP-Methode	GET
Pfad	/journal/entries/{id}
Authentifizierung	JWT

Path-Parameter

Parameter	Typ	Pflicht	Beschreibung
id	int	ja	Primärschlüssel des Eintrags

HTTP-Statuscodes

Code	Bedeutung
200	Erfolgreiche Verarbeitung
401	Fehlende oder ungültige Authentifizierung
422	Validierungsfehler (Pydantic / FastAPI)
500	Unerwarteter Serverfehler
404	Eintrag nicht gefunden oder gehört nicht zum Nutzer

4.4.3 POST /journal/entries/ - Eintrag erstellen

Feld	Wert
Beschreibung	Erstellt einen neuen Eintrag. Triggert anschließend asynchron die KI-Anreicherung via BackgroundTask (enrich_journal_entry).
HTTP-Methode	POST
Pfad	/journal/entries/
Authentifizierung	JWT

Request-Body

```
{
  "title": "Reflections on My Day",
  "date": "2026-05-16",
  "content": "Today was productive. I finished a major task and took a long walk.",
  "sentiment_level": 4,
  "sleep_quality": 3,
  "stress_level": 2,
  "social_engagement": 3
}
```

Validierungsregeln

- Pflichtfelder: title, date, content.
- Skalen-Werte (sentiment_level, sleep_quality, stress_level, social_engagement) sind IntEnum 1-5, optional.
- date als ISO-8601 Date (YYYY-MM-DD).
- title, content als Strings ohne explizite Längenbegrenzung (Empfehlung: serverseitig ergänzen).

Response (200)

```
{
  "id": 23,
  "title": "Reflections on My Day",
  "date": "2026-05-16",
  "content": "Today was productive...",
  "sentiment_level": 4,
  "sleep_quality": 3,
  "stress_level": 2,
  "social_engagement": 3,
  "formatted_content": "Today was productive...",
  "activities": "",
  "sentiments": "",
  "created_at": "2026-05-22T10:14:33Z",
  "updated_at": null,
  "goals": []
}
```

HTTP-Statuscodes

Code	Bedeutung
200	Erfolgreiche Verarbeitung
401	Fehlende oder ungültige Authentifizierung
422	Validierungsfehler (Pydantic / FastAPI)

Code	Bedeutung
500	Unerwarteter Serverfehler

Fachliche Business-Logik

- Synchron: INSERT in `journal_entries` mit `user_id`, gesetzten Skalen-Enums (`.value`), `formatted_content` initial = `content`.
- Asynchron (BackgroundTask): `get_goal_info(user_id)` holt alle Ziele -> `analyze_entry(content, goals)` ruft das LLM auf.
- Das LLM liefert `formatted_content`, `activities`, `sentiments`, `goal_ids`. Diese werden gesetzt, `get_goals(goal_ids)` verknüpft sie via Association-Tabelle.
- `updated_at` wird auf den Anreicherungszeitpunkt gesetzt; AI-Aufruf erzeugt `ai_usage_logs`-Eintrag.

Beispielablauf

- Client sendet POST `/journal/entries/` mit Bearer-Token.
- Server validiert Token (Cognito JWKS), lädt/legt User an.
- CRUD speichert Eintrag, Response wird sofort an Client zurückgegeben.
- BackgroundTask startet, ruft Gemini/z.ai mit Eintrag und Ziel-Kontext auf.
- Eintrag wird mit KI-Resultaten aktualisiert (`formatted_content`, `activities`, `sentiments`, `goals`).
- Bei Folgeabruf (GET) erhält der Client die angereicherten Daten.

4.4.4 PUT `/journal/entries/{id}` - Eintrag aktualisieren

Feld	Wert
Beschreibung	Aktualisiert einen vorhandenen Eintrag. Triggert die AI-Anreicherung erneut, falls sich <code>content</code> geändert hat.
HTTP-Methode	PUT
Pfad	<code>/journal/entries/{id}</code>
Authentifizierung	JWT

Request-Body (alle Felder optional)

```
{
  "title": "Reflections on My Day",
  "date": "2026-05-16",
  "content": "Today was productive.",
  "sentiment_level": 4,
  "sleep_quality": 3,
  "stress_level": 4,
  "social_engagement": 3
}
```

Fachliche Business-Logik

- Lookup Eintrag inkl. `user_id`-Filter (Mandantentrennung).
- `model_dump(exclude_unset=True)` -> Partial Update.
- `updated_at = now(UTC)`.
- Wenn `content` im Update enthalten war: erneute Background-Anreicherung.

HTTP-Statuscodes

Code	Bedeutung
200	Erfolgreiche Verarbeitung
401	Fehlende oder ungültige Authentifizierung
422	Validierungsfehler (Pydantic / FastAPI)
500	Unerwarteter Serverfehler
404	Eintrag nicht gefunden

Bekannter Defekt (Inferiert)

In `app/db/crud/journal.py` liefert `update_journal_entry` am Ende einen Ausdruck der Form `'return db_entry, content_changed if db_entry else (None, False)'`.

Aufgrund Python-Operator-Präzedenz bezieht sich der ternäre Ausdruck nur auf `content_changed`, nicht auf das Tupel. Bei `db_entry = None` würde der Aufrufer dennoch `(None, False)` erwarten, erhält aber tatsächlich `(None, False)`, weil `content_changed` dann `False` ist - das Verhalten ist aktuell korrekt, aber rein zufällig.

Empfehlung: explizite Klammerung `'return (db_entry, content_changed) if db_entry else (None, False)'` für Lesbarkeit und Robustheit.

4.4.5 DELETE /journal/entries/{id} - Eintrag löschen

Feld	Wert
Beschreibung	Hartes Löschen des Eintrags. Verknüpfungen in <code>journal_goal_association</code> sind über SQLAlchemy-Cascading nicht explizit konfiguriert (siehe Kapitel 9).
HTTP-Methode	DELETE
Pfad	<code>/journal/entries/{id}</code>
Authentifizierung	JWT

Response (200)

```
{ "message": "Journal entry with id 7 deleted successfully" }
```

HTTP-Statuscodes

Code	Bedeutung
200	Erfolgreiche Verarbeitung
401	Fehlende oder ungültige Authentifizierung
422	Validierungsfehler (Pydantic / FastAPI)
500	Unerwarteter Serverfehler
404	Eintrag nicht gefunden

4.5 Goal-Endpoints

4.5.1 GET /goals/ - Ziele listen

Feld	Wert
Beschreibung	Listet alle Ziele des Nutzers, sortiert nach Priorität (High, Medium, Low). Verknüpfte Einträge werden auf max. 5 jüngste reduziert (max_entries_per_goal).
HTTP-Methode	GET
Pfad	/goals/
Authentifizierung	JWT

Query-Parameter

Parameter	Typ	Pflicht	Beschreibung
skip	int (>=0)	nein	Offset, Default 0
limit	int (1-100)	nein	Limit, Default 100

4.5.2 GET /goals/{id} - Einzelnes Ziel

Feld	Wert
Beschreibung	Lädt ein Ziel inklusive max. 5 jüngsten verknüpften Einträgen. 404 bei Nicht-Existenz / Fremdzugriff.
HTTP-Methode	GET
Pfad	/goals/{id}
Authentifizierung	JWT

4.5.3 POST /goals/ - Ziel erstellen

Feld	Wert
Beschreibung	Legt ein neues Ziel an. progress wird initial mit 0 gesetzt.
HTTP-Methode	POST
Pfad	/goals/
Authentifizierung	JWT

Request-Body

```
{
  "title": "Take long walks",
  "type": "Recurring",
  "category": "Health",
  "priority": "Medium",
  "description": "Taking long walks is good for your health.",
  "targetDate": "2026-12-31"
}
```

Validierungsregeln

- Pflichtfelder: title, type, category, priority.

- priority ist Enum 'High' | 'Medium' | 'Low'.
- targetDate (alias target_date) als ISO-Date, optional.
- populate_by_name = True erlaubt sowohl 'targetDate' (Frontend-Convention) als auch 'target_date'.

4.5.4 PUT /goals/{id} - Ziel aktualisieren

Feld	Wert
Beschreibung	Partielles Update eines Ziels inklusive optionalem progress (0-100).
HTTP-Methode	PUT
Pfad	/goals/{id}
Authentifizierung	JWT

Request-Body (Auszug)

```
{
  "title": "Complete Python Course",
  "type": "One-time",
  "target_date": "2026-06-30",
  "category": "Education",
  "priority": "Medium",
  "description": "Finish the Python course.",
  "progress": 60
}
```

Fachliche Business-Logik

- model_dump(exclude_unset=True) -> Partial Update.
- priority wird als Enum-Value (.value) gespeichert.
- updated_at = now(UTC).
- Hinweis: Es existiert keine serverseitige Validierung, ob progress in [0, 100] liegt; sollte ergänzt werden.

4.5.5 DELETE /goals/{id}

Feld	Wert
Beschreibung	Hartes Löschen eines Ziels.
HTTP-Methode	DELETE
Pfad	/goals/{id}
Authentifizierung	JWT

Response (200)

```
{ "message": "Goal with id 7 deleted successfully" }
```

4.5.6 POST /goals/recommend - KI-Zielvorschläge

Feld	Wert
Beschreibung	Liest die jüngsten Einträge, konsolidiert deren Inhalte und liefert eine LLM-generierte Liste empfohlener Ziele.

Feld	Wert
HTTP-Methode	POST
Pfad	/goals/recommend
Authentifizierung	JWT

Response (200)

```
[
  {
    "title": "Develop a weekly stress management routine",
    "description": "Set aside time each week for activities that reduce stress.",
    "category": "Health"
  }
]
```

Fachliche Business-Logik

- Aggregation der entry.content-Felder zu einem Prompt-Kontext.
- LLM-Aufruf recommend_goals -> JSON-Parse -> RecommendedGoalList.
- AI-Log wird erzeugt; bei 0 Einträgen leere Liste.

4.5.7 POST /goals/enhance-description

Feld	Wert
Beschreibung	Verbessert oder generiert eine Zielbeschreibung anhand des Titels und einer optionalen Bestehenden.
HTTP-Methode	POST
Pfad	/goals/enhance-description
Authentifizierung	JWT

Request-Body

```
{
  "title": "Run 5 km without stopping",
  "description": "Start training plan"
}
```

Response (200)

```
"Build endurance step by step with a structured plan, starting with short intervals and gradually progressing to a continuous 5 km run."
```

4.6 AI-Endpoints

4.6.1 POST /ai/chat/ - Kontextbezogener Chatbot

Feld	Wert
Beschreibung	Sendet eine Nutzernachricht an einen LLM-Assistenten mit System-Prompt und Nutzerkontext (max. 10 Ziele + 10 jüngste Einträge).
HTTP-Methode	POST
Pfad	/ai/chat/
Authentifizierung	JWT

Request-Body

```
{ "message": "How have I been feeling this week?" }
```

Response (200)

```
{ "text": "Looking back at your last entries, you sound..." }
```

Fachliche Business-Logik

- System-Prompt definiert Tonalität, Datenschutz-Regeln und Funktionen des Assistenten.
- Kontext-Block enthält Goals (Titel, Priorität, Target, Beschreibung) und Einträge (Datum, Titel, Content).
- Antwort wird als JSON mit Feld 'text' erwartet (response_format json_object).

4.6.2 POST /ai/journal-question/ - Reflexionsfrage

Feld	Wert
Beschreibung	Generiert eine offene, einzelne Reflexionsfrage zu einem in Arbeit befindlichen Eintrag.
HTTP-Methode	POST
Pfad	/ai/journal-question/
Authentifizierung	JWT

Request-Body

```
{ "content": "Heute war anstrengend, aber ich habe viel geschafft." }
```

Response (200)

```
{ "question": "Welcher Moment hat dich heute am meisten beansprucht?" }
```


4.7 Analytics-Endpoints

4.7.1 GET /analytics/trends/ - Zeitreihen-Trends

Feld	Wert
Beschreibung	Liefert Moving-Average-Zeitreihen für Sentiment, Sleep, Stress, Social über die gewählte Periode.
HTTP-Methode	GET
Pfad	/analytics/trends/
Authentifizierung	JWT

Query-Parameter

Parameter	Typ	Pflicht	Beschreibung
period	string	nein	z. B. '7days', '30days', '6months', '1years'. Default '30days'.

Response (200)

```
{
  "dates": ["2026-04-22", "2026-04-23", "..."],
  "sentiment": [4, 4, 3, "..."],
  "sleep": [3, 3, 4, "..."],
  "stress": [2, 2, 3, "..."],
  "social": [3, 3, 3, "..."]
}
```

Fachliche Business-Logik

- Filterung der Einträge auf $\geq$ cutoff_date (UTC).
- Window-Size automatisch: $\min(21, \max(3, \text{past_days} // 7))$.
- Fehlende Skalen-Werte werden mit Default 3 ersetzt (Neutralwert).
- Ausgabe als gerundete Integer (round).

4.7.2 GET /analytics/stats/ - Durchschnitte und KPIs

Feld	Wert
Beschreibung	Liefert Durchschnitte, Gesamtzahl der Einträge, aktuellen Streak und durchschnittliche Wortanzahl pro Eintrag.
HTTP-Methode	GET
Pfad	/analytics/stats/
Authentifizierung	JWT

Response (200)

```
{
  "sentiment": 3.7,
  "sleep": 3.2,
  "stress": 2.8,
  "social": 3.1,
  "total_entries": 24,
}
```

```
{
  "current_streak": 5,
  "average_words_per_entry": 142.3
}
```

Fachliche Business-Logik

- Durchschnitte ignorieren NULL-Werte (None) auf den Skalen.
- current_streak: zählt rückwärts ab heute (UTC-Date), solange entry.date in der Menge enthalten ist.
- Lookback für Streak-Berechnung: max. 365 Tage.

4.7.3 GET /analytics/correlations/ - Korrelationen + KI-Insights

Feld	Wert
Beschreibung	Berechnet Pearson-Korrelationen zwischen Skalen, liefert die 2 stärksten und erzeugt im Hintergrund KI-Insights.
HTTP-Methode	GET
Pfad	/analytics/correlations/
Authentifizierung	JWT

Response (200, gekürzt)

```
{
  "strongest_correlations": {
    "sleep_sentiment": {
      "correlation": 0.72,
      "data": [ {"date": "2026-05-01", "x": 4, "y": 4}, "..."],
      "x_label": "Sleep Quality",
      "y_label": "Sentiment Level",
      "x_avg": [3.3, 3.5, "..."],
      "y_avg": [3.5, 3.8, "..."],
      "insights": ["Better sleep correlates with...", "Tip: ...", "Action: ..."]
    }
  },
  "total_data_points": 18
}
```

Fachliche Business-Logik

- Aufbau eines aligned-Datensatzes nur für Einträge mit allen vier Skalenwerten.
- Berechnung von 6 Paarkorrelationen via numpy.corrcoef.
- Sortierung nach |r| desc, Auswahl der Top-2.
- Cache-Lookup: bei Hit (TTL 24h) Sofortantwort, sonst Stub mit leeren insights + BackgroundTask zur Berechnung.
- Bei < 2 Datenpunkten: Hinweis-Message statt Daten.

4.7.4 GET /analytics/summary/ - KI-Zusammenfassung

Feld	Wert
Beschreibung	Generiert (oder liest aus Cache) eine narrative Zusammenfassung der Periode mit emotionalen Mustern und Themen.
HTTP-Methode	GET

Feld	Wert
Pfad	/analytics/summary/
Authentifizierung	JWT

Response (200)

```
{ "summary": "Over the past entries, you reflected deeply on..." }
```

Fachliche Business-Logik

- Cache-Hit: Rückgabe aus AnalyticsCache.
- Cache-Miss: Stub-Response 'Generating insights... Check back shortly.' + BackgroundTask.
- Background: prepare_summary_data -> summarize_journal_entries -> _set_cache.
- Bei 0 Einträgen: explizite Meldung 'No journal entries found for the specified period.'

4.8 Admin-Endpoints

4.8.1 GET /admin/stats

Feld	Wert
Beschreibung	Liefert systemweite Kennzahlen zur Plattform. Erfordert is_admin = true im User-Record.
HTTP-Methode	GET
Pfad	/admin/stats
Authentifizierung	JWT + Admin-Rolle

Response (200)

```
{
  "total_users": 312,
  "active_users_7d": 84,
  "total_entries": 4218,
  "total_ai_calls": 9341,
  "ai_success_rate": 98.7,
  "ai_calls_today": 142,
  "ai_tokens_total": 1820431
}
```

Fachliche Business-Logik

- active_users_7d: distinct user_id aus journal_entries mit created_at >= now() - 7d.
- ai_success_rate = (success=true) / total_ai_calls * 100, gerundet auf 1 Nachkommastelle.
- ai_calls_today: created_at >= today_start (00:00 UTC).
- ai_tokens_total: COALESCE(SUM(input_tokens), 0) + COALESCE(SUM(output_tokens), 0).

HTTP-Statuscodes

Code	Bedeutung
200	Erfolgreiche Verarbeitung
401	Fehlende oder ungültige Authentifizierung
422	Validierungsfehler (Pydantic / FastAPI)
500	Unerwarteter Serverfehler
403	Authentifiziert, aber nicht Admin

5. Business-Logik und Zustandsmodell

5.1 Zentrale Geschäftsprozesse

- Eintrags-Lifecycle: Erstellung -> KI-Anreicherung -> optionale Updates -> Löschung.
- Ziel-Lifecycle: Anlage -> Pflege (Beschreibung, Priorität, progress) -> Verknüpfung mit Einträgen via KI -> Abschluss/Löschung.
- Analytics-Lifecycle: Anfrage -> Cache-Lookup -> Direktantwort oder Stub + Background-Berechnung -> Cache-Schreiben.
- AI-Aufrufzyklus: Aufruf -> Retry (bei 429/5xx) -> Erfolg/Fehler -> Logging in ai_usage_logs.

5.2 Zustand JournalEntry

Der Eintrag besitzt keinen expliziten Status-Spalten, jedoch implizite Zustände, abgeleitet aus den befüllten Anreicherungsfeldern.

Zustand (implizit)	Charakteristik
CREATED	Eintrag persistiert, formatted_content == content, activities und sentiments leer, goals leer.
ENRICHING	BackgroundTask läuft. Datenbank-seitig nicht direkt sichtbar (keine Status-Spalte).
ENRICHED	formatted_content angereichert, activities und sentiments befüllt, ggf. goals verknüpft, updated_at != null.
ENRICHMENT_FAILED	Logged in App-Logger, Eintrag verbleibt im CREATED-Zustand. Kein Retry-Mechanismus.

5.2.1 Zustandsübergänge

Aktueller Zustand	Trigger	Neuer Zustand
(nicht existent)	POST /journal/entries/	CREATED
CREATED	BackgroundTask startet	ENRICHING
ENRICHING	LLM-Aufruf erfolgreich	ENRICHED
ENRICHING	LLM-Aufruf scheitert (nach Retries)	ENRICHMENT_FAILED
ENRICHED	PUT mit Content-Änderung	ENRICHING
ENRICHED	PUT ohne Content-Änderung	ENRICHED (nur Update der Skalen)
beliebig	DELETE /journal/entries/{id}	(nicht existent)

5.3 Zustand Goal

Ziele haben keinen ausgewiesenen Lifecycle-Status. Der einzig dynamische Wert ist progress (0-100). Eine fachliche Klassifikation kann anhand von progress + target_date abgeleitet werden:

Inferierter Status	Regel
ACTIVE	progress < 100 und (target_date null oder >= heute)

Inferierter Status	Regel
COMPLETED	progress == 100
OVERDUE	progress < 100 und target_date < heute

Diese Klassifikation ist im Code nicht persistiert; sie sollte für UI-Zwecke entweder im Backend abgeleitet oder als zusätzliches Feld eingeführt werden.

5.4 Zustand AnalyticsCache

Zustand	Definition	Verhalten des Endpoints
MISS	Kein Record vorhanden	Stub-Response + BackgroundTask zur Erstberechnung
FRESH	Record vorhanden, generated_at innerhalb TTL 24h	Direktantwort aus Cache
STALE	Record vorhanden, generated_at älter als 24h	Aktuell als MISS behandelt - kein Aged-Cache-Return + asynchroner Refresh

Empfehlung: STALE separat behandeln (Stale-While-Revalidate), um bessere UX bei abgelaufenem Cache zu erzielen.

5.5 Validierungslogik

- Eingangsvalidierung über Pydantic-Schemas (JournalEntryCreate, GoalCreate, etc.).
- Enum-Bindung der Skalen (IntEnum 1-5) verhindert Out-of-Range-Werte.
- Pfad-Parameter id: int wird von FastAPI typisiert geprüft.
- Mandantentrennung: Jeder CRUD-Lookup filtert auf user_id; Fremdzugriffe führen zu 404.
- Nicht abgedeckt: Längenbeschränkungen für title/content/description, progress-Range, target_date in der Zukunft.

5.6 Fehlerfälle

Fehlerfall	Verhalten
Ungültiges/abgelaufenes JWT	HTTP 401 'Invalid or expired token', WWW-Authenticate-Header
Fehlender Token	HTTP 401 'Not authenticated'
Admin-Endpoint ohne Admin-Rechte	HTTP 403 'Admin access required'
Ressource gehört nicht zum Nutzer	HTTP 404 (nicht 403) - bewusst keine Existenz-Leakage
Pydantic-Validierungsfehler	HTTP 422 mit Feld-Details
AI-Aufruf RateLimit / 5xx	Bis zu 3 Retries mit Exponential Backoff (1s, 2s, 4s)
AI-Aufruf endgültig gescheitert	AIUsageLog mit success=false; Background-Task loggt Fehler
Cache-Schreibfehler	Wird im Background-Task geloggt, Endpoint-Antwort bleibt unberührt

5.7 Abhängigkeiten zwischen Prozessen

- Goal-Zuordnung in Einträgen hängt von der Existenz aktiver Ziele beim AI-Aufruf ab. Wenn Ziele nach der Anreicherung gelöscht werden, bleibt die Association-Tabelle dangling, bis sie explizit aufgeräumt wird (siehe Kapitel 9).
- Analytics setzen auf das Feld `entry.sentiments` (Komma-Liste) auf. Nicht angereicherte Einträge erscheinen nicht in der `prepare_summary_data`.
- Streak-Berechnung ist robust gegen fehlende Anreicherung, da sie nur auf `entry.date` abstellt.
- Goal Recommendations setzen auf rohen content auf, nicht auf angereicherten Inhalt.

6. Sequenz- und Datenflussbeschreibungen

6.1 Konvention textueller Sequenzdiagramme

- Akteure sind links (Client) -> Server-Komponenten (Router, Service, CRUD, AI, DB) nach rechts.
- Notation: Akteur -> Komponente : Aktion; Rückgaben mit '<-'.

6.2 Sequenz: Eintrag erstellen mit Hintergrund-Anreicherung

```

Client -> JournalRouter : POST /journal/entries/ (JWT, JSON-Body)
JournalRouter -> AuthDep : verify token + load user
AuthDep -> CognitoJWKS : GET /.well-known/jwks.json (TTL 1h)
CognitoJWKS <- AuthDep : keys
AuthDep -> UsersDB : SELECT user by cognito_sub
UsersDB <- AuthDep : user (or insert if missing)
JournalRouter -> JournalCRUD : create_journal_entry(entry_id, user_id)
JournalCRUD -> EntriesDB : INSERT journal_entries
EntriesDB <- JournalCRUD : db_entry
JournalRouter -> BackgroundTask : enrich_journal_entry(entry_id, user_id)
JournalRouter <- Client : 200 OK (JournalEntry without enrichment)

# Background path
BackgroundTask -> JournalCRUD : get_journal_entry
BackgroundTask -> GoalsDB : get_goal_info
BackgroundTask -> AIClient : analyze_entry(content, goals)
AIClient -> Gemini/Z.AI : chat.completions.create (json_object)
AIClient -> AIUsageLogsDB : INSERT ai_usage_logs (success/fail)
BackgroundTask -> EntriesDB : UPDATE formatted_content, activities, sentiments, goals

```

6.3 Sequenz: Korrelationen abrufen

```

Client -> AnalyticsRouter : GET /analytics/correlations/?period=30days
AnalyticsRouter -> AuthDep : verify token, load user
AnalyticsRouter -> CacheDB : SELECT analytics_cache (user, period,
'correlation_insights')
CacheDB <- AnalyticsRouter : cached or null

# Cache-Hit
AnalyticsRouter <- Client : 200 OK (JSON.parse(cached))

# Cache-Miss
AnalyticsRouter -> Analytics : calculate_correlations_data(past_days)
Analytics -> EntriesDB : SELECT entries WHERE date >= cutoff
Analytics <- AnalyticsRouter : result with top-2 correlations (insights = [])
AnalyticsRouter -> BackgroundTask : _generate_correlations_background
AnalyticsRouter <- Client : 200 OK (stub with empty insights)

# Background
BackgroundTask -> AIClient : generate_correlation_insights(chart_data)
BackgroundTask -> CacheDB : INSERT/UPDATE analytics_cache

```

6.4 Sequenz: Chatbot-Antwort


```

Client -> ChatbotRouter      : POST /ai/chat/ {message}
ChatbotRouter -> AuthDep      : verify token, load user
ChatbotRouter -> GoalCRUD     : get_goals(user_id, limit=10)
ChatbotRouter -> JournalCRUD  : get_journal_entries(user_id, limit=10)
ChatbotRouter -> ChatbotService : get_contextual_chatbot_response(message, context)
ChatbotService -> AIClient    : call_ai(messages=[system, user], feature='chatbot')
AIClient -> Gemini/Z.AI       : chat.completions.create
AIClient -> AIUsageLogsDB     : INSERT ai_usage_logs
ChatbotRouter <- Client      : 200 OK {text}

```

6.5 Sequenz: Admin-Statistiken

```

Client -> AdminRouter        : GET /admin/stats
AdminRouter -> AuthDep        : verify token, get_current_admin
AuthDep -> UsersDB            : SELECT user
AuthDep validates            : user.is_admin == true (else 403)
AdminRouter -> UsersDB        : COUNT(users)
AdminRouter -> EntriesDB      : COUNT(distinct user_id WHERE created_at>=7d)
AdminRouter -> EntriesDB      : COUNT(entries)
AdminRouter -> AIUsageLogsDB  : COUNT, SUM(input_tokens), SUM(output_tokens)
AdminRouter <- Client        : 200 OK { aggregated stats }

```

6.6 Datenflüsse extern

Externes System	Datenfluss
AWS Cognito	Backend lädt JWKS (HTTPS) -> verifiziert Signatur lokal. Es werden keine sensiblen User-Daten zurückgeschrieben.
Gemini / Z.AI	Backend sendet Prompts (Journal-Texte, Ziele, Skalen). Antwort = JSON. Token-Counts werden in ai_usage_logs persistiert.
Frontend (localhost:3000)	Reine HTTPS/REST-Kommunikation. CORS ist explizit auf den Origin beschränkt.

Datenschutz-Hinweis (Inferiert): Da Journal-Inhalte zu AI-Providern gesendet werden, ist eine explizite Auftragsverarbeitungs-Vereinbarung (AVV) bzw. DPA mit dem Provider erforderlich, sofern produktiv mit personenbezogenen Daten gearbeitet wird.

7. Sicherheits- und Architekturkonzept

7.1 Authentifizierung

- Mechanismus: OAuth2 Bearer Token (JWT) ausgestellt durch AWS Cognito User Pool.
- Algorithmus: RS256, asymmetrische Signaturverifikation via JWKS.
- Issuer/Audience-Validierung: cognito-idp.<region>.amazonaws.com/<user_pool_id> und audience = COGNITO_APP_CLIENT_ID.
- JWKS-Cache: TTLCache(maxsize=1, ttl=3600). Reduziert Round-Trips zu Cognito.
- Token-Schema: OAuth2PasswordBearer(tokenUrl='auth/token', auto_error=False) - der Token-Endpoint ist im aktuellen Code nicht implementiert (siehe Kapitel 9).

7.2 Autorisierung und Rollenmodell

Rolle	Rechte
Anonym	Zugriff nur auf GET / (Healthcheck).
Authentifizierter Nutzer	CRUD nur auf eigene Einträge, Ziele und Analytics; alle /auth/me, /journal, /goals, /ai, /analytics-Endpoints.
Admin (is_admin=true)	Zusätzlich Zugriff auf /admin/stats und perspektivisch weitere Admin-Endpoints (laut ERD GSI2: ENTITY#<type>-Lookups).

Die Admin-Privilegisierung erfolgt händisch über das Skript scripts/make_admin.py oder direkt in der Datenbank. Es existiert kein Self-Service-Pfad zur Promotion - das ist sicherheitstechnisch korrekt.

7.3 Datenvalidierung

- Eingabevalidierung an der Routing-Grenze über Pydantic v2.
- Enum-Bindung verhindert ungültige Skalenwerte.
- Path-/Query-Parameter werden via FastAPI Query() mit ge/le-Constraints geprüft.
- Empfehlung: max_length für title/content/description ergänzen, regex-basierte Sanitization, Schutz gegen exzessiv lange LLM-Prompts.

7.4 Fehlerbehandlung und Logging

- FastAPI liefert strukturierte 4xx-/5xx-Responses mit JSON-Body.
- App-eigenes Logging über Python logging-Modul; Verwendung in services/analytics.py, services/ai_client.py, db/crud/journal.py.
- AI-Aufrufe werden vollständig in ai_usage_logs auditiert (Feature, Modell, Tokens, success, error_message).
- Empfehlung: zentrales Exception-Handler-Middleware (FastAPI exception_handler) für einheitliche Fehler-Responses und Korrelations-IDs.

7.5 Sicherheitsrisiken

Risiko	Schweregrad	Beschreibung
DEV_MODE aktiv in Produktion	Hoch	DEV_MODE=true umgeht jede Authentifizierung und nutzt einen

Risiko	Schweregrad	Beschreibung
		Admin-Dev-User. Risiko bei falscher Env-Konfiguration im Deployment.
AI-Datenübermittlung	Hoch	Journal-Inhalte (potenziell sensibel) verlassen das System. Erfordert DPA, ggf. EU-Region und Anonymisierung.
Fehlender Token-Endpoint	Mittel	OAuth2PasswordBearer verweist auf 'auth/token', der nicht implementiert ist. Login-Flow erfolgt extern (Cognito Hosted UI) - sollte dokumentiert sein.
Hartes Löschen	Mittel	Kein Soft-Delete, kein Audit-Trail. Datenwiederherstellung nur via Backup.
Fehlende Rate-Limits	Mittel	Keine Begrenzung der Anfragen pro Nutzer/IP. Risiko Brute-Force, Cost-Abuse via AI-Endpoints.
SQLite-Konkurrenz	Mittel	Concurrent writes können in produktiver Last zu Locking-Problemen führen.
CORS-Whitelist statisch	Niedrig	http://localhost:3000 - in Prod muss die Liste konfigurierbar sein (env-basiert).
Fehlende Längen-Caps	Niedrig	Lange Texte könnten LLM-Kosten und Speicher steigern.
Keine Goal-Range-Validierung	Niedrig	progress lässt sich auf negative oder >100 Werte setzen.

7.6 Schutzmaßnahmen (Empfohlen)

- DEV_MODE explizit nur bei lokaler Entwicklung; Production-Deployment-Pipeline darf DEV_MODE nicht setzen. Optional: Boot-time-Refusal bei DEV_MODE + non-localhost.
- Auftragsverarbeitungs-Vereinbarung mit AI-Provider, Logging-Opt-out, ggf. Selbsthosting via OpenAI-kompatibler Open-Source-Modelle.
- Rate-Limiting via SlowAPI oder API-Gateway (z. B. AWS API Gateway, traefik, nginx).
- Migration auf PostgreSQL/DynamoDB für produktive Konkurrenz.
- Einführung Soft-Delete (deleted_at) für reversible Löschungen.
- Sicherheitsheader (HSTS, X-Content-Type-Options, etc.) via Middleware setzen.
- Secrets-Verwaltung über AWS Secrets Manager / SSM Parameter Store, nicht .env in Containern.

8. Technische Bewertung der Architektur

8.1 Skalierbarkeit

Dimension	Bewertung
Horizontal	FastAPI ist stateless, Skalierung über mehrere Replicas grundsätzlich möglich. BackgroundTasks laufen jedoch im Worker-Prozess und gehen bei Pod-Restart verloren - kein Queue.
Vertikal	Begrenzt durch SQLite-Single-Writer-Lock. Bei höheren Schreibraten Bottleneck.
AI-Throughput	Limitiert durch Provider-Quotas. Retry-Logik puffert kurzfristige Rate Limits, aber Backoff sequenziell.
Caching	AnalyticsCache reduziert Provider-Last sinnvoll; JWKS-Cache spart Cognito-Roundtrips.

8.2 Wartbarkeit

- Saubere Modulgliederung (routes / services / models / db / utils) erleichtert Onboarding.
- Pydantic-Schemas dienen gleichzeitig als Doku (OpenAPI) und Validierungslayer.
- Schwach: Inkonsistenz zwischen ERD-Spezifikation (DynamoDB) und Code (SQLAlchemy) erschwert architekturelle Konsistenzprüfung.
- Schwach: keine automatisierten Tests im Repo sichtbar - hohes Regressions-Risiko bei Refactorings.
- Schwach: Bruno-Sammlung enthält veraltete Pfade (analytics/averages, weekly-patterns) - Quelle der Wahrheit nicht eindeutig.

8.3 Erweiterbarkeit

- AI-Provider-Wechsel ist über Environment-Variable AI_PROVIDER möglich. Sauber abstrahiert.
- Neue Endpoints lassen sich nach dem Schema route -> service -> crud sauber ergänzen.
- Datenmodell hinreichend allgemein, um zusätzliche Skalen oder Tagging-Mechanismen aufzunehmen.
- Erschwert: Migration zu DynamoDB wäre tief eingreifend (kein ORM, Access-Pattern-zentriertes Design).

8.4 Kopplung und Kohäsion

- Routen sind dünn und delegieren überwiegend an Services / CRUD (Loose Coupling).
- Services kapseln klare Verantwortlichkeiten (AI, Analytics, Chatbot).
- Schwach: services/gemini_agent.py und services/gemini_chatbot.py heißen nach Provider Gemini, sind aber providerunabhängig - Naming-Schuld.
- Schwach: db/crud/journal.py enthält direkten Import auf services/gemini_agent.py - leichte Vermischung von Persistenz- und KI-Layer (cohesion gut, coupling erhöht).

8.5 Technische Risiken

Risiko	Auswirkung
Architektur-Drift (ERD vs. Code)	Stakeholder-Verwirrung, Migrationen drohen teuer; Spezifikation muss vereinheitlicht werden.
Keine sichtbaren Tests	Regressionen bei Änderungen, schwierige Release-Qualifizierung.
BackgroundTasks ohne Queue	Verlust laufender Anreicherungen bei Neustart, keine Retries auf Task-Ebene.
Hardcoded Modellnamen ('gemma-4-31b-it', 'GLM-5-Turbo')	Diese Modell-Strings wirken erfunden / nicht offiziell - Risiko, dass Aufrufe gegen Provider fehlschlagen. Sollte verifiziert / konfigurierbar gemacht werden.
SQLite in Produktion	Concurrency-Probleme, fehlende Replikation, kein Cluster-Failover.
Background-Failure ist still	Fehlgeschlagene Enrichments werden nur geloggt, kein Nutzer-Feedback und kein Retry.

8.6 Verbesserungspotenziale

- Test-Suite einführen: pytest + httpx.AsyncClient, Fokus auf Auth-Negativtests, CRUD-Mandantentrennung, AI-Mocking.
- Background-Verarbeitung auf Celery/RQ/AWS SQS umstellen, mit Retries und Dead-Letter-Queue.
- Persistenzschicht harmonisieren: entweder vollständig auf PostgreSQL umstellen oder ERD-konform auf DynamoDB migrieren.
- Statusfeld für JournalEntry einführen (CREATED / ENRICHING / ENRICHED / FAILED) - aktuell implizit.
- Soft-Delete + Audit-Felder (deleted_at, deleted_by).
- Einheitliches Error-Handling via FastAPI exception_handler mit Correlation-IDs.
- Observability: Prometheus-Metriken, strukturierte JSON-Logs, OpenTelemetry-Traces.
- OpenAPI-Doku ergänzen: Beispiele, Tags, response_descriptions, security_schemes.
- Bruno-Sammlung an Code synchronisieren oder durch generierte httpie/postman-Collection ersetzen.

9. Offene Fragen und Annahmen

9.1 Fehlende Informationen

- Soll-Datenmodell: Welches Modell ist verbindlich, SQLAlchemy/SQLite (Ist) oder DynamoDB-Single-Table (ERD)? Beide produktiv parallel nicht haltbar.
- AI-Provider und Modellnamen: 'gemma-4-31b-it' und 'GLM-5-Turbo' im Code sind ungewöhnlich - sind das Platzhalter, Tippfehler oder echte Modellbezeichnungen?
- Token-Endpoint: OAuth2PasswordBearer verweist auf '/auth/token', der nicht existiert. Wie loggt sich der Client ein? Vermutlich über Cognito Hosted UI - sollte explizit dokumentiert sein.
- Frontend-Konfiguration: Wird das Frontend ausschließlich auf localhost:3000 betrieben? Production-Domain muss in CORS-Liste.
- Cron / Scheduled Jobs: Gibt es geplante Wartungs-Jobs (alte Caches bereinigen, Streak-Recompute)?

9.2 Architekturannahmen

- Annahme: SQLite ist Entwicklungs-/Demo-Setup, produktive Persistenz ist offen.
- Annahme: BackgroundTasks sind ausreichend für die aktuelle Last; eine Queue ist nicht notwendig.
- Annahme: Cognito-User Pool ist die einzige Identity-Source (kein Multi-IdP).
- Annahme: Das Frontend kennt das DTO-Schema und mappt targetDate <-> target_date korrekt.
- Annahme: Skalenwerte 1-5 sind ausreichend granular; keine kontinuierlichen Werte vorgesehen.

9.3 Potenzielle Risiken

Risiko	Empfohlene Gegenmaßnahme
DEV_MODE in Produktion	Boot-Refusal außerhalb localhost / explizite Flagprüfung im Deployment
Dangling Associations bei Goal-Löschung	Cascade-Konfiguration oder explizites Cleanup in CRUD
Datenleck via AI-Provider	DPA prüfen, ggf. Self-Hosted-LLM, sensible Daten redacten
Cache-Race-Conditions	INSERT/UPDATE per UNIQUE-Constraint absichern, Upsert atomar
Tippfehler bei AI-Modellnamen	Konfigurierbar machen, Fail-Fast beim Boot, wenn Modell nicht erreichbar
Background-Failure ohne Sichtbarkeit	Status-Feld am Eintrag, Retry-Strategie, Admin-Sicht über fehlgeschlagene Tasks

9.4 Unklare Anforderungen

- Progress-Tracking: Soll progress automatisch berechnet werden (z. B. aus verknüpften Einträgen) oder nur manuell gepflegt?
- Recurring Goals: Wie wird die 'Wiederkehr' fachlich modelliert? Aktuell ist 'type' freier String.

- Mehrsprachigkeit: System-Prompts sind englisch, Beispieldaten teils englisch, teils deutsch - ist Lokalisierung geplant?
- Datenexport / DSGVO-Auskunft: Gibt es einen Endpoint, der einem Nutzer alle seine Daten exportiert?
- Account-Löschung: Wie wird DSGVO Art. 17 (Recht auf Löschung) umgesetzt? Aktuell kein Endpoint dafür sichtbar.

9.5 Zusammenfassung Soll/Ist-Klärungsbedarf

Vor Übergang in den produktiven Betrieb sollte ein architektureller Klärungsworkshop erfolgen, der die folgenden vier Kernfragen adressiert:

- 1. Datenbank-Strategie: DynamoDB (ERD) oder relational (Ist)?
- 2. Authentifizierungs-Flow: Wie wird Login dokumentiert und gegen DEV_MODE abgesichert?
- 3. AI-Provider-Verbindlichkeit: Welcher Provider, welches Modell, welche DPA?
- 4. Background-Verarbeitung: Bleibt BackgroundTasks oder wird Queue eingeführt?

Die Beantwortung dieser Fragen ist Voraussetzung für eine belastbare Produktiv-Architektur und bildet die Grundlage für die nächste Iteration des Fachkonzepts (Version 2.0).